

EXPERIMENT 3

Shravanya Andhale
D20A-02

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

THEORY:

A **cryptocurrency** is a digital form of currency that can use Blockchain technology to record and verify transactions without depending entirely on a central authority. In this experiment, a simple cryptocurrency system is implemented using Python. The Blockchain is simulated using three independent nodes running on different ports: **5001, 5002, and 5003**. Each node maintains its own copy of the Blockchain and communicates with other nodes through a peer-to-peer network.

The major concepts implemented in this experiment are:

1. Blockchain and Blocks:

Each node maintains a chain of blocks containing cryptocurrency transactions. A block generally consists of an index, timestamp, list of transactions, proof or nonce, and the hash of the previous block. The first block is called the genesis block. Each new block is connected to the previous block using its hash, forming a secure chain of records.

2. Cryptocurrency Transactions:

A transaction represents the transfer of cryptocurrency from one participant to another. Each transaction contains information such as the **sender, receiver, and amount**.

Transactions are first stored as pending transactions before they are included in a block. For example, a transaction can represent a transfer of 5 units from user a to user b.

3. Mempool:

The **mempool** is the temporary storage area for transactions that have been created but have not yet been included in a mined block. When a transaction is submitted, it is added to the pending transaction list. During mining, the miner collects these pending transactions and adds them to a new block. After the transactions are successfully included in a block, they should be removed from the pending transaction list to prevent the same transactions from being added again.

4. Mining:

Mining is the process of creating a new block and adding it to the Blockchain. The miner collects pending transactions and performs the Proof-of-Work process to find a valid proof or nonce. Once the required condition is satisfied, the new block is created and added to the Blockchain. In a cryptocurrency system, the miner can also receive a reward transaction for successfully mining a block.

5. Proof-of-Work:

Proof-of-Work is a consensus mechanism that requires the miner to perform

computational work before a block can be added to the Blockchain. The program repeatedly searches for a suitable proof value until the cryptographic condition is satisfied. Once a valid proof is found, the block can be mined and added to the chain. Other nodes can verify the proof without repeating the complete mining process.

6. **Peer-to-Peer Network:**

In this experiment, three Blockchain nodes are used to simulate a peer-to-peer network. The nodes run independently on ports **5001, 5002, and 5003**. Each node maintains its own copy of the Blockchain and can communicate with other nodes using HTTP requests. The `connect_node()` function is used to register the addresses of other nodes as peers. This allows the nodes to communicate and synchronize their Blockchain data.

7. **Consensus and Longest Chain Rule:**

Since different nodes may temporarily have different versions or lengths of the Blockchain, a consensus mechanism is required. In this experiment, the **Longest Chain Rule** is used. When a node receives information about other chains, it checks whether a longer valid chain exists. If a longer valid Blockchain is found, the node replaces its current chain with the longer chain. This helps the nodes eventually maintain a consistent copy of the Blockchain.

8. **Chain Replacement:**

The `replace_chain()` function is used to synchronize a node with its peers. When this function is called, the node communicates with the connected nodes and obtains their Blockchain information. It compares the chain lengths and validates the chains received from other nodes. If a longer valid chain is available, the current node replaces its chain with that chain. This demonstrates how Blockchain nodes can resolve differences between their ledgers.

9. **Transaction and Block Synchronization:**

Transactions are initially created on a particular node and stored in its pending transaction list. After mining, the transactions become part of a block. The Blockchain can then be shared between connected nodes. If another node has a shorter chain, it can use the chain replacement mechanism to obtain the longer valid chain. In this way, the Blockchain can be synchronized across the simulated network.

10. **Flask API:**

Flask is used to create the web API through which the Blockchain nodes can be accessed. Each node runs a Flask application on a different port. API requests can be sent using **Postman** to perform operations such as connecting nodes, adding transactions, mining blocks, fetching the Blockchain, and replacing the chain. This makes it possible to demonstrate the working of the cryptocurrency and Blockchain through HTTP requests.

11. **Libraries Used:**

Python libraries are used to implement different functions of the system. `datetime` is used for generating timestamps, `hashlib` is used for SHA-256 hashing and Proof-of-Work, and `uuid4` can be used to generate a unique identifier for a node. Flask's `request` and `jsonify`

are used to receive request data and return JSON responses. `urlparse` is used for processing node URLs, while the `requests` library is used for communication between different Blockchain nodes.

The three-node setup demonstrates the basic working of a decentralized cryptocurrency network. Each node can independently maintain a Blockchain, accept transactions, mine blocks, and communicate with other nodes. The use of Proof-of-Work provides a mining mechanism, while the Longest Chain Rule helps the nodes reach a common Blockchain when their copies differ.

CODE AND OUTPUT:

```
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP3> python -m venv venv
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP3>
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP3> Set-ExecutionPolicy -S
cope Process -ExecutionPolicy Bypass
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP3> .\env\Scripts\Activat
e.ps1
(venv) PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP3> pip install re
quests==2.18.4
Collecting requests==2.18.4
  Downloading requests-2.18.4-py2.py3-none-any.whl (88 kB)
    |████████████████████| 88 kB 790 kB/s
Collecting chardet<3.1.0,>=3.0.2
  Downloading chardet-3.0.4-py2.py3-none-any.whl (133 kB)
    |████████████████████| 133 kB 6.8 MB/s
Collecting idna<2.7,>=2.5
  Downloading idna-2.6-py2.py3-none-any.whl (56 kB)
    |████████████████████| 56 kB 2.0 MB/s
Collecting certifi>=2017.4.17
  Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
    |████████████████████| 136 kB 3.3 MB/s
Collecting urllib3<1.23,>=1.21.1
  Downloading urllib3-1.22-py2.py3-none-any.whl (132 kB)
    |████████████████████| 132 kB 6.4 MB/s
Installing collected packages: urllib3, idna, chardet, certifi, requests
Successfully installed certifi-2026.7.22 chardet-3.0.4 idna-2.6 requests-
2.18.4 urllib3-1.22
WARNING: You are using pip version 21.2.3; however, version 26.2.1 is av
ailable.
You should consider upgrading via the 'C:\Users\Shravanya\Desktop\BLOCKCH
AIN_LAB\EXP3\venv\Scripts\python.exe -m pip install --upgrade pip' comman
```

REST API basics: CRUD, test & variable / Get data

SaveShare

GET

http://127.0.0.1:5001/get_chain

Send

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (5)

Test Results (1/1)

200 OK • 72 ms • 290 B • Save Response

{ } JSON

Preview

Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:26:02.262301",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

Overview

GET Get data

No environment

REST API basics: CRUD, test & variable / Get data

SaveShare

GET

http://127.0.0.1:5002/get_chain

Send

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (5)

Test Results (1/1)

200 OK • 11 ms • 290 B • Save Response

{ } JSON

Preview

Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:31:31.796449",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

REST API basics: CRUD, test & variable / **Get data** Save Share

GET http://127.0.0.1:5003/get_chain Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results (1/1) 200 OK • 13 ms • 290 B Save Response

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:31:43.627787",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

POST http://127.0.0.1:5001/connect_node Send

Params Authorization Headers (8) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5002",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results (1/1) 201 CREATED • 12 ms • 267 B Save Response

JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected.",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5002"
6   ]
7 }
```

POST http://127.0.0.1:5002/connect_node Send

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results (1/1) 201 CREATED • 9 ms • 267 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected.",
3   "total_nodes": [
4     "127.0.0.1:5001",
5     "127.0.0.1:5003"
6   ]
7 }
```

POST http://127.0.0.1:5003/connect_node Send

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Body Cookies Headers (5) Test Results (1/1) 201 CREATED • 8 ms • 325 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following
3     nodes:",
4   "total_nodes": [
5     "127.0.0.1:5002",
6     "127.0.0.1:5001"
7   ]
8 }
```

REST API basics: CRUD, test & variable / **Post data** Save Share

POST ▼ Send ▼

Params Authorization Headers (8) **Body** ● Scripts ● Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 {
2   "sender": "shravanya",
3   "receiver": "sanket",
4   "amount": 51
5 }
```

Body Cookies Headers (5) Test Results (1/1) 🕒 **201 CREATED** • 9 ms • 226 B • 🌐 Save Response ⋮

{} **JSON** ▼ ▶ Preview 🖼 Visualize ▼ 🔍 🔗

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
```

POST ▼ Send ▼

Params Authorization Headers (8) **Body** ● Scripts ● Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 {
2   "sender": "shravanya",
3   "receiver": "srushti",
4   "amount": 52
5 }
```

Body Cookies Headers (5) Test Results (1/1) 🕒 **201 CREATED** • 8 ms • 226 B • 🌐 Save Response ⋮

{} **JSON** ▼ ▶ Preview 🖼 Visualize ▼ 🔍 🔗

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
```

POST http://127.0.0.1:5001/add_transaction

Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "sender": "shravanya",
3   "receiver": "simba",
4   "amount": 53
5 }
```

Body Cookies Headers (5) Test Results (1/1) 201 CREATED 11 ms 226 B Save Response

{ JSON Preview Visualize

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
```

GET http://127.0.0.1:5001/mine_block

Params Authorization Headers (6) Body Scripts Settings

Body Cookies Headers (5) Test Results (1/1) 200 OK

{ JSON Preview Visualize

```
2 "index": 2,
3 "message": "Congratulations, you just mined a block!",
4 "previous_hash": "108ee43d8bc7b9dbaaccbf69a7e1951b64d82bf502ad71b3ed324da9874d505",
5 "proof": 533,
6 "timestamp": "2026-09-02 12:37:58.613900",
7 "transactions": [
8   {
9     "amount": 51,
10    "receiver": "sanket",
11    "sender": "shravanya"
12  },
13  {
14    "amount": 52,
15    "receiver": "siushti",
16    "sender": "shravanya"
17  },
18  {
19    "amount": 53,
20    "receiver": "simba",
21    "sender": "shravanya"
22  },
23  {
24    "amount": 1,
25    "receiver": "Shravanya",
26    "sender": "f7b1a7aa35f74b7b84772661d1aceb01"
```

GET http://127.0.0.1:5001/mine_block

Params Authorization Headers (6) Body Scripts • Settings

Body Cookies Headers (5) Test Results (1/1) ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "index": 3,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "3087042c732536fde22e1b878b236b48aaec2217e2870abaced78841cb62268b",
5   "proof": 45293,
6   "timestamp": "2026-09-02 12:38:49.957538",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Shravanya",
11      "sender": "f7b1a7aa35f74b7b84772661d1aceb01"
12    }
13  ]
14 }
```

GET http://127.0.0.1:5001/get_chain

Params Authorization Headers (6) Body Scripts • Settings

Body Cookies Headers (5) Test Results (1/1) ↻ 200 OK

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:26:02.262301",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "108ee43d8bc7b9dbaaccbf69a7e1951b64d82bf502ad71b3ed324da9874d505",
13      "proof": 533,
14      "timestamp": "2026-09-02 12:37:58.613900",
15      "transactions": [
16        {
17          "amount": 51,
18          "receiver": "sanket",
19          "sender": "shravanya"
20        },
21        {
22          "amount": 52,
23          "receiver": "siushti",
24          "sender": "shravanya"
25        }
26      ]
27    }
28  ]
29 }
```

```

    "index": 3,
    "previous_hash": "3087042c732536fde22e1b878b236b48aaec2217e2870abaced78841cb62268b",
    "proof": 45293,
    "timestamp": "2026-09-02 12:38:49.957538",
    "transactions": [
      {
        "amount": 1,
        "receiver": "Shravanya",
        "sender": "f7b1a7aa35f74b7b84772661d1aceb01"
      }
    ]
  },
  {
    "index": 4,
    "previous_hash": "c0cd49ac856019e389d0edd4c8e2bf269109fb394bb3da859af656b14672ae29",
    "proof": 21391,
    "timestamp": "2026-09-02 12:39:04.409479",
    "transactions": [
      {
        "amount": 1,
        "receiver": "Shravanya",
        "sender": "f7b1a7aa35f74b7b84772661d1aceb01"
      }
    ]
  }
]

```

GET
http://127.0.0.1:5002/get_chain

Params
Authorization
Headers (6)
Body
Scripts
Settings

Body
Cookies
Headers (5)
Test Results (1/1)

{} JSON
Preview
Visualize

```

1  {
2    "chain": [
3      {
4        "index": 1,
5        "previous_hash": "0",
6        "proof": 1,
7        "timestamp": "2026-09-02 12:31:31.796449",
8        "transactions": []
9      }
10   ],
11   "length": 1
12 }

```

GET http://127.0.0.1:5003/get_chain

Params Authorization Headers (6) Body Scripts Settings

Body Cookies Headers (5) Test Results (1/1)

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:31:43.627787",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
```

GET http://127.0.0.1:5002/replace_chain

Params Authorization Headers (6) Body Scripts Settings

Body Cookies Headers (5) Test Results (1/1) 200 OK

{ } JSON Preview Visualize

```
2 "message": "The nodes had different chains so the chain was replaced by the longest one.",
3 "new_chain": [
4   {
5     "index": 1,
6     "previous_hash": "0",
7     "proof": 1,
8     "timestamp": "2026-09-02 12:26:02.262301",
9     "transactions": []
10  },
11  {
12    "index": 2,
13    "previous_hash": "108ee43d8bc7b9dbaaccbf69a7e1951b64d82bf502ad71b3ed324da9874d505",
14    "proof": 533,
15    "timestamp": "2026-09-02 12:37:58.613900",
16    "transactions": [
17      {
18        "amount": 51,
19        "receiver": "sanket",
20        "sender": "shravanya"
21      },
22      {
23        "amount": 52,
24        "receiver": "srushti",
25        "sender": "shravanya"
26      }
27    ]
28  }
29 ]
```

```

    {
      "index": 4,
      "previous_hash": "c0cd49ac856019e389d0edd4c8e2bf269109fb394bb3da859af656b14672ae29",
      "proof": 21391,
      "timestamp": "2026-09-02 12:39:04.409479",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Shravanya",
          "sender": "f7b1a7aa35f74b7b84772661d1aceb01"
        }
      ]
    }
  ],
  "message": "All good. The chain is the largest one."
}

```

GET http://127.0.0.1:5003/replace_chain

Params Authorization Headers (6) Body Scripts Settings

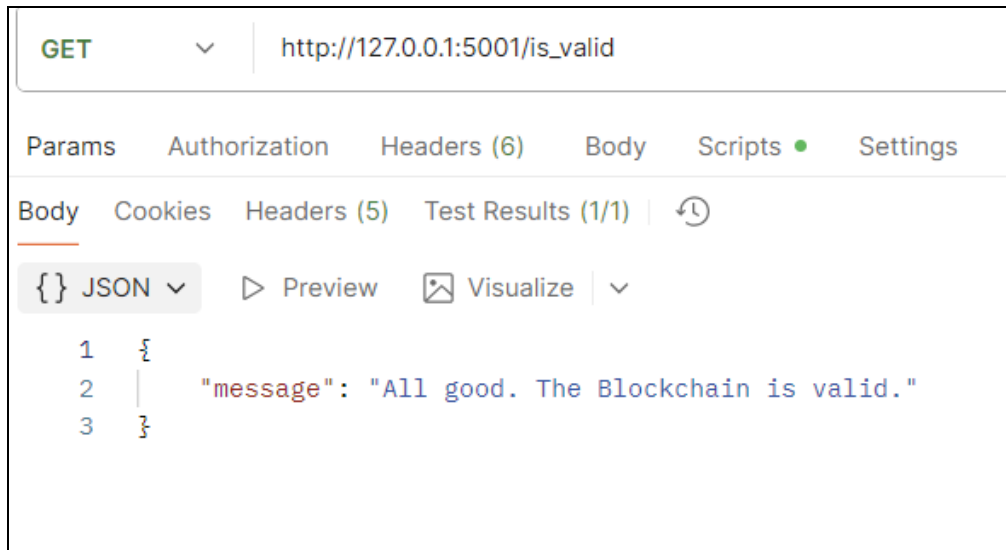
Body Cookies Headers (5) Test Results (1/1) 200 OK

{ } JSON Preview Visualize

```

1  {
2    "message": "The nodes had different chains, so the chain was replaced by the longest one.",
3    "new_chain": [
4      {
5        "index": 1,
6        "previous_hash": "0",
7        "proof": 1,
8        "timestamp": "2026-09-02 12:26:02.262301",
9        "transactions": []
10     },
11     {
12       "index": 2,
13       "previous_hash": "108ee43d8bc7b9dbaaccbf69a7e1951b64d82bf502ad71b3ed324da9874d505",
14       "proof": 533,
15       "timestamp": "2026-09-02 12:37:58.613900",
16       "transactions": [
17         {
18           "amount": 51,
19           "receiver": "sanket",
20           "sender": "shravanya"
21         },
22         {

```



CONCLUSION:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.